

GHANA INSTITUTE OF MANAGEMENT AND PUBLIC ADMINISTRATION

School of Technology | Department of Computer Science

CS 305: Problem Set 2 (Grammars & Non-Regular Languages)

Weeks 4–7 | Total Marks: 100 | Weight: 7.5% of Final Grade

Instructions & Guidelines

- Pumping Lemma proofs must follow the step-by-step adversarial template presented in lecture.
- CFG derivations must state all variable substitution rules explicitly.
- Pushdown Automata transitions must specify stack read/write operations (e.g., $a, b \rightarrow c$).

Problem 1: Pumping Lemma Proofs for Regular Languages (25 Marks)

Using the **Pumping Lemma for Regular Languages** (Sipser Theorem 1.70), rigorously prove that the following languages are **not regular**:

(a) $L_1 = \{0^n 1^m 0^n \mid n, m \geq 0\}$. (12 Marks)

(b) $L_2 = \{ww \mid w \in \{a, b\}^*\}$. (13 Marks)

Problem 2: Context-Free Grammar Design & Ambiguity (25 Marks)

Let $\Sigma = \{a, b\}$.

- (a) Construct a Context-Free Grammar G that generates the language $L_3 = \{a^i b^j \mid i \neq j \text{ and } i, j \geq 0\}$. (12 Marks)
- (b) Consider the arithmetic grammar G_{expr} with rules: $E \rightarrow E + E \mid E \times E \mid (E) \mid a$.
- (i) Prove that G_{expr} is **ambiguous** by providing two distinct leftmost derivations (or two distinct parse trees) for the string $s = a + a \times a$. (7 Marks)
- (ii) Rewrite G_{expr} into an equivalent unambiguous grammar enforcing standard operator precedence ($\times$ over $+$). (6 Marks)

Problem 3: Pushdown Automata (PDA) Construction (25 Marks)

Let $\Sigma = \{0, 1\}$. Design a formal Pushdown Automaton $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ that recognizes the language:

$$L_4 = \{w \in \{0, 1\}^* \mid w \text{ has equal number of 0's and 1's}\}$$

- (a) State the formal components $(Q, \Sigma, \Gamma, q_0, F)$. (10 Marks)
- (b) Draw the complete PDA state diagram labeling all transition inputs and stack actions $a, b \rightarrow c$. (15 Marks)

Problem 4: Modern Applied Case Study — Grammar-Guided LLM JSON Decoding (25 Marks)

Modern generative AI inference frameworks (e.g., HuggingFace `outlines`, Microsoft `guidance`) force LLMs to output syntactically valid JSON objects by intersecting vocabulary token logits with a Context-Free Grammar mask at runtime.

- (a) Write a simple CFG G_{JSON} over terminal alphabet $\Sigma = \{\{, \}, \text{key}, :, \text{val}, ,\}$ generating key-value JSON objects with arbitrary nesting depth. (12 Marks)
- (b) Explain why a finite state machine (DFA) cannot validate arbitrary recursive JSON nesting depth, and how a pushdown parser mask guarantees that the LLM will never produce a malformed JSON syntax error. (13 Marks)